CAMPAIGN INTEGRITY

Low-Level Design & Critical Feature Specification

Track: Martech Approval
Team: MAL 4 · 1 Nov 2026
Architecture Deliverable

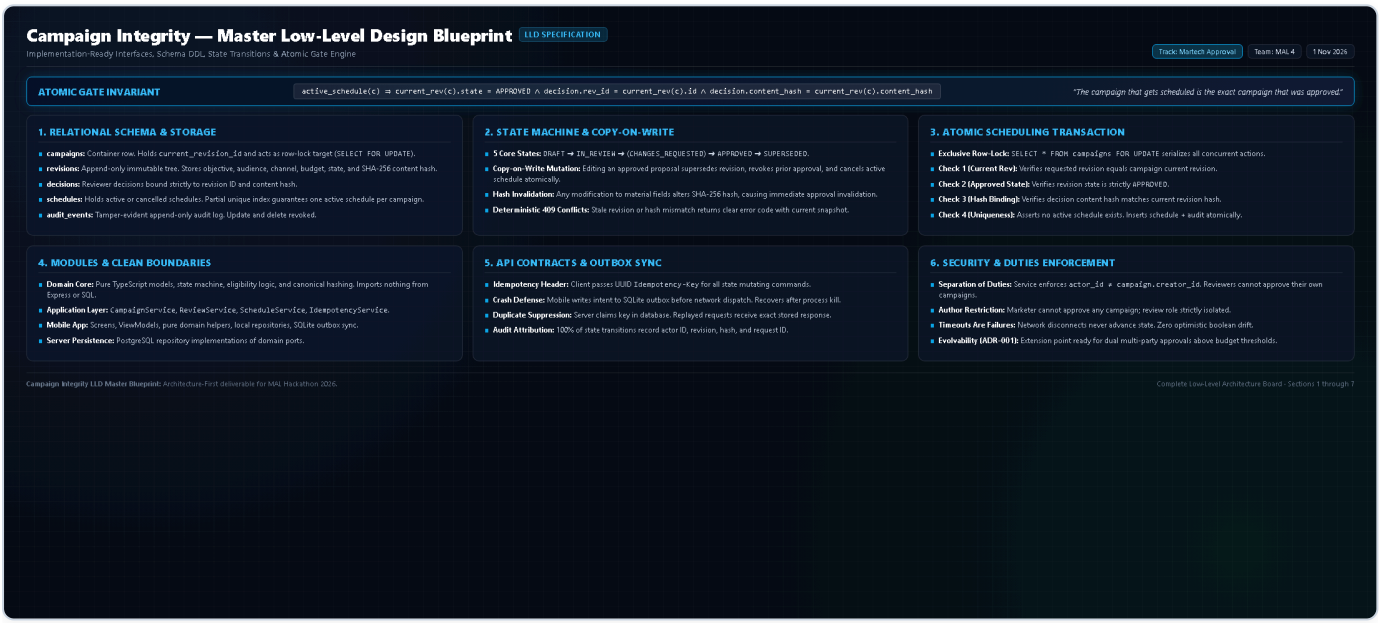
THE CORE ARCHITECTURAL INVARIANT

```
active_schedule(c) ⇒ current_rev(c).state = APPROVED ∧ decision.rev_id = current_rev(c).id ∧
decision.content_hash = current_rev(c).content_hash
```

"The campaign that gets scheduled is the exact campaign that was approved."

1 Executive Architectural Blueprint

This document specifies the concrete modules, interfaces, models, states, database schemas, and the atomic scheduling transaction for **Campaign Integrity**. The design eliminates boolean status drift by binding every approval cryptographically and relationally to an immutable revision tree.



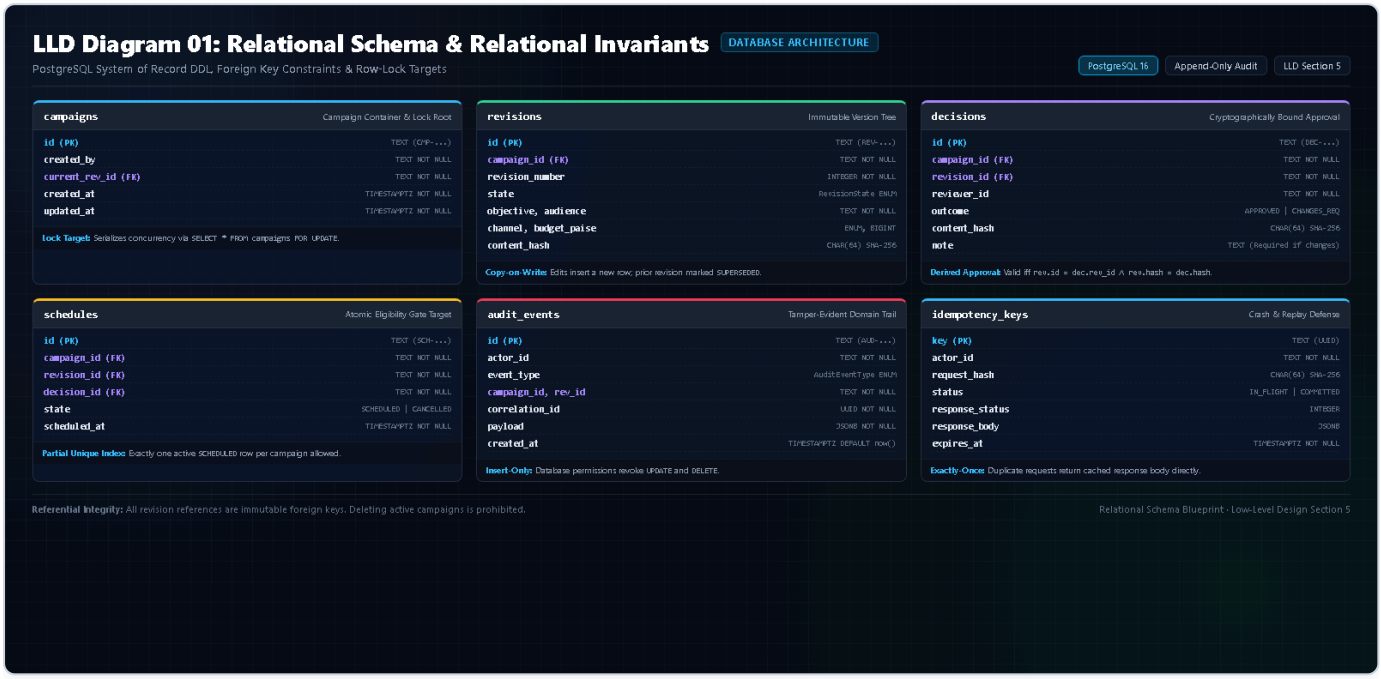
2 Clean Architecture Module Map

LAYER	RESPONSIBILITIES	MAY IMPORT	FORBIDDEN IMPORTS
Mobile Presentation	Campaign dashboard, revision comparisons, review drawer, action triggers	View models, Design System	SQLite, HTTP client, Server code
Mobile Application	Orchestrates reads, enqueues commands, manages local form states	Domain types, Repository ports	React Native UI components
Domain Core	Pure state machine, eligibility predicates, canonical SHA-256 hash	Pure TypeScript only	Express, pg, React Native, SQLite
Application Services	Transaction boundaries, command authorization, audit trail emission	Domain models, Repository ports	Express request/response objects
Server Persistence	SQL queries, row locks (`SELECT FOR UPDATE`), constraint mapping	Domain types, pg driver	HTTP layer, UI layer

RELATIONAL SCHEMA & DATA MODELS

PostgreSQL System of Record DDL & Integrity Constraints

LLD Section 5



3 Relational Table Specifications

TABLE	PRIMARY KEY	KEY FOREIGN KEYS & CONSTRAINTS	OPERATIONAL INVARIANT
campaigns	id (CMP-...)	current_rev_id ➔ revisions(id)	Root lock target for all concurrent reviews and schedule gates.
revisions	id (REV-...)	campaign_id ➔ campaigns(id)	Append-only immutable revision tree. Content hash computed over material fields.
decisions	id (DEC-...)	revision_id ➔ revisions(id)	Bound to exact revision SHA-256 content hash. Cannot exist without revision target.
schedules	id (SCH-...)	revision_id, decision_id	Partial unique index: exactly ONE active SCHEDULED row per campaign.
audit_events	id (AUD-...)	correlation_id (UUID)	Tamper-evident append-only log. UPDATE and DELETE revoked at database level.
idempotency_keys	key (UUID)	request_hash (SHA-256)	Deduplication engine. Replays cached response body for network retries.

4 Canonical Content Hashing (SHA-256)

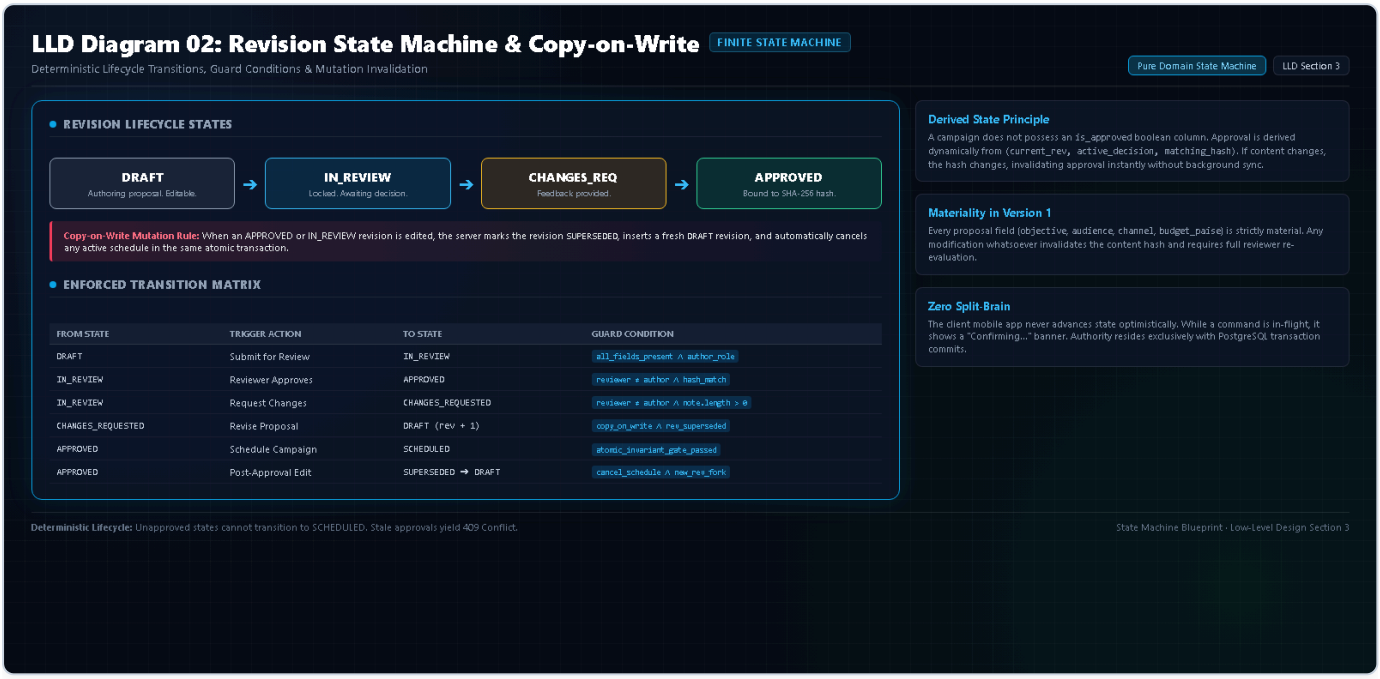
Every proposal revision computes a canonical SHA-256 hash across material fields to guarantee bit-level integrity:

```
function computeCanonicalHash(rev: ProposalContent): string { const payload = JSON.stringify({ objective: rev.objective.trim(), audience: rev.audience.trim(), channel: rev.channel, budgetPaise: rev.budgetPaise.toString() }); return crypto.createHash("sha256").update(payload, "utf8").digest("hex"); }
```

REVISION STATE MACHINE & ATOMIC GATE

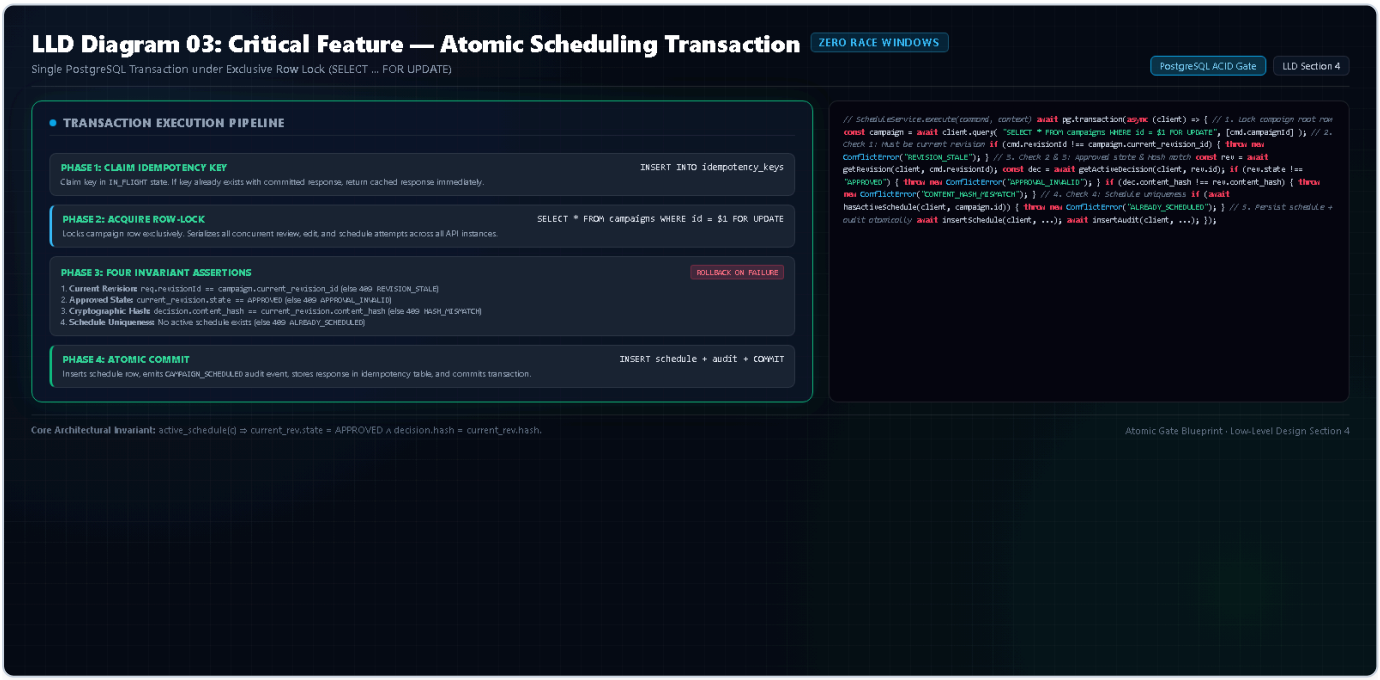
Deterministic Transitions & The Critical Feature Transaction

LLD Section 3 & 4



5 Critical Feature: Atomic Scheduling Transaction

Scheduling is evaluated within a single PostgreSQL ACID transaction under exclusive row-lock (`SELECT ... FOR UPDATE`). Zero race windows exist between concurrent edits and reviewer actions:



6 Transaction Implementation (TypeScript)

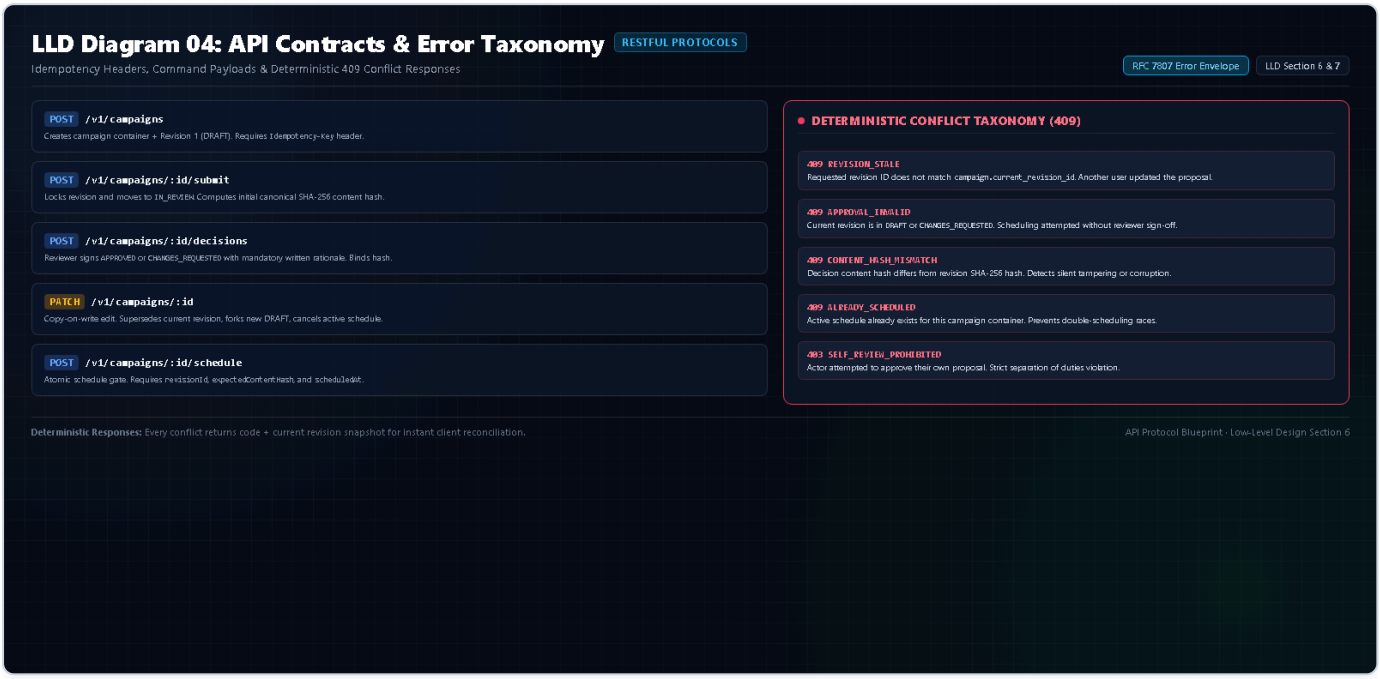
```
export class ScheduleService { async schedule(cmd: ScheduleCommand, ctx: RequestContext): Promise<ScheduleResult> { return this.pg.transaction(async (tx) => { // 1. Acquire exclusive lock on campaign root const campaign = await tx.query("SELECT * FROM campaigns WHERE id = $1 FOR UPDATE", [cmd.campaignId]); // Check 1: Current revision check if (cmd.revisionId !== campaign.current_revision_id) { throw new ConflictError("REVISION_STALE", { current: campaign.current_revision_id }); } // Check 2: Approved state check const rev = await this.revRepo.getById(tx, cmd.revisionId); if (rev.state !== "APPROVED") { throw new ConflictError("APPROVAL_INVALID", { state: rev.state }); } // Check 3: Cryptographic content hash equality const dec = await
```

```
this.decRepo.getActiveDecision(tx, rev.id); if (dec.contentHash !== rev.contentHash) { throw new
ConflictError("CONTENT_HASH_MISMATCH"); } // Check 4: Schedule uniqueness check if (await this.schedRepo.hasActiveSchedule(tx,
campaign.id)) { throw new ConflictError("ALREADY_SCHEDULED"); } // Commit schedule and audit event atomically const schedule = await
this.schedRepo.create(tx, { campaignId: campaign.id, revId: rev.id, decId: dec.id }); await this.auditRepo.emit(tx, { type:
"CAMPAIGN_SCHEDULED", campaignId: campaign.id, revId: rev.id, actorId: ctx.actorId }); return schedule; }); } }
```

API PROTOCOLS & RUNTIME RESILIENCE

Idempotent Commands, Conflict Envelopes & Offline Outbox

LLD Section 6 & 7



7 RESTful API Command Specifications

ENDPOINT	METHOD	IDEMPOTENCY	KEY VALIDATION	DOMAIN EFFECT
`/v1/campaigns`	POST	Required	Schema & fields	Creates campaign + Revision 1 (DRAFT)
`/v1/campaigns/:id/submit`	POST	Required	Author role	Locks Rev 1; state moves to IN_REVIEW
`/v1/campaigns/:id/decisions`	POST	Required	`reviewer ≠ author`	Inserts decision bound to SHA-256 hash; state ➔ APPROVED
`/v1/campaigns/:id`	PATCH	Required	Author role	Copy-on-write fork: Rev 2 DRAFT created; cancels active schedule!
`/v1/campaigns/:id/schedule`	POST	Required	Row-lock gate	Evaluates 4 checks; inserts schedule + audit row atomically

8 Defensibility Matrix & Quality Guarantees

QUALITY ATTRIBUTE	CONCRETE FAILURE SCENARIO	ARCHITECTURAL DEFENSE MECHANISM
Integrity	Marketer alters budget post-approval and attempts schedule.	Copy-on-write supersedes revision; hash mismatch rejects schedule via atomic gate.
Recoverability	App crashes or loses connectivity mid-command.	Client SQLite outbox retains intent; background sync retries with identical UUID key.
Concurrency	Reviewer approves while marketer submits simultaneous edit.	PostgreSQL row lock (`SELECT FOR UPDATE`) on campaign serializes execution.
Auditability	Auditor inspects historical decision for regulatory review.	Append-only `audit_events` logs actor, timestamp, revision ID, hash, and request ID.